



دانشگاه فردوسی مشهد
گروه مهندسی کامپیوتر

گزارش پروژه درس
مبانی هوش محاسباتی

Persian Sentiment Analysis with Ensemble Learning

تحلیل احساسات نظرات فارسی با یادگیری ترکیبی

نام و نام خانوادگی	شماره دانشجویی
امیرحسین ابوالفضلی	4022262035
آرمان بیجاری	4021262131

استاد: دکتر فضل ارثی

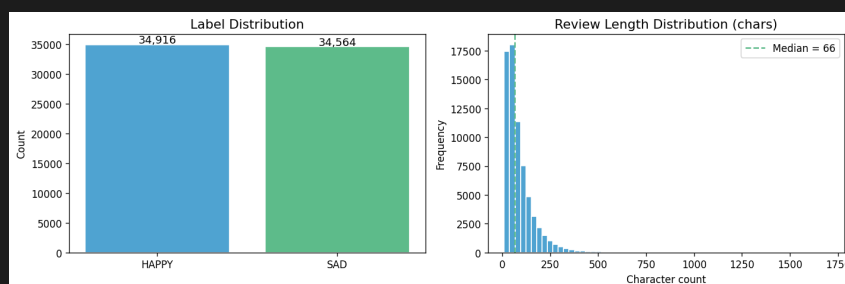
نیمسال دوم 1404-1405

فهرست مطالب

۲	۱ معرفی داده‌ها
۲	۲ فاز اول: پیش‌پردازش متن
۲	۱.۲ چرا فارسی سخت‌تره؟
۲	۲.۲ مراحل پیش‌پردازش
۳	۳.۲ نمونه‌هایی از قبل و بعد
۳	۴.۲ تقسیم داده
۳	۳ فاز دوم: Vectorization
۴	۱.۳ CountVectorizer
۴	۲.۳ TF-IDF
۴	۳.۳ Word2Vec
۵	۴ فاز سوم: آموزش و ارزیابی مدل‌های پایه
۵	۱.۴ مدل‌ها و تنظیماتشون
۶	۲.۴ نتایج کامل
۸	۳.۴ تحلیل نتایج
۸	۴.۴ نمونه‌هایی از پیش‌بینی‌های درست و اشتباه
۹	۵ فاز چهارم: انتخاب بهترین Vectorizer
۹	۱.۵ مقایسه میانگین عملکرد
۱۰	۲.۵ چرا CountVectorizer بهتر از TF-IDF شد؟
۱۰	۶ فاز پنجم: پیاده‌سازی Stacking
۱۰	۱.۶ طراحی
۱۱	۲.۶ نتایج
۱۱	۳.۶ تحلیل
۱۲	۴.۶ نمونه‌هایی از پیش‌بینی‌های درست و اشتباه مدل نهایی
۱۲	۷ نتیجه‌گیری

۱. معرفی داده‌ها

داده‌ای که توی این پروژه باهاش کار کردیم Snapfood Persian Sentiment Analysis هستش که از Kaggle^۱ برداشتیم. داده حدوداً 70,000 تا نظر کاربری Snapfood داره که کاربرها بعد از سفارش غذا نوشتن. هر نظر یه برچسب داره: یا HAPPY (مثبت) یا SAD (منفی). خوبی این داده اینه که کاملاً متوازنه، یعنی تعداد نظرات مثبت و منفی تقریباً 35,000 تا 35,000 تاست و مشکل عدم توازن کلاس نداریم. یه چیزی که داده رو جالب و در عین حال چالش برانگیز می‌کنه اینه که نظرات کاملاً واقعی و عامیانه‌ان. یعنی کاربرها غلط نگارشی دارن، بعضیا حروف رو کشیدن مثل «عایی» که بخوان تأکید کنن، بعضیا emoji گذاشتن، بعضیا هم کلمه فارسی و انگلیسی رو قاطی کردن. این چیزها همه نویز حساب میشن و باید قبل از هر کاری پاک بشن.



شکل ۱: توزیع برچسب‌های مثبت و منفی در داده

پس داده خوبیه، ولی چون عامیانه و غیررسمیه، پیش‌پردازش درست خیلی مهمه.

۲. فاز اول: پیش‌پردازش متن

۱.۲. چرا فارسی سخت‌تره؟

یه مشکل خاص که فارسی داره اینه که بعضی حروفش با عربی گُد یونیکد مشترک ندارن ولی عین هم به نظر میان. مثلاً «ی» فارسی کدش U+06CC هستش ولی «ي» عربی کدش U+064A هستش. از نظر بصری فرقی نداری ولی وقتی مدل می‌خواد یه vocabulary^۲ بسازه، این دوتا رو دوتا کلمه جدا می‌بینه. پس کلمه «می‌خوام» که یه بار با «ی» فارسی نوشته شده و یه بار با «ي» عربی، از دید مدل دو چیز کاملاً مجزاست. به خاطر همین نرمال‌سازی اول از همه باید انجام بشه.

۲.۲. مراحل پیش‌پردازش

کد پیش‌پردازش توی فایل src/preprocessing.py نوشته شده. ترتیب مراحل مهمه چون هر مرحله پیش‌شرط مرحله بعدیه.

اول از همه URL‌ها رو حذف می‌کنیم. این مرحله باید قبل از نرمال‌سازی باشه، چون Hazm^۳ داخل فرایند نرمال‌سازیش کاراکتر «/» رو حذف می‌کنه و بعدش دیگه نمیشه لینک رو تشخیص داد.

^۱Kaggle: پلتفرم آنلاین برای مسابقات یادگیری ماشین و به اشتراک‌گذاری دیتاست.

^۲vocabulary: مجموعه همه کلمات منحصر به فردی که مدل باهاشون آشناست و می‌تونه بردار بسازه.

^۳Hazm: کتابخانه پایتون برای پردازش زبان طبیعی فارسی. امکانات نرمال‌سازی، توکن‌سازی، ریشه‌یابی و لیست کلمات بی‌معنی رو

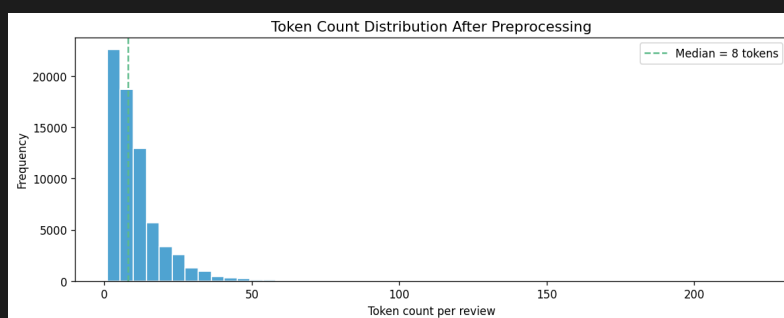
داره.

یه مرحله‌ای که به نظرمون جالبه مرحله کاهش تکراره. الگوی `r'\{2,\}'` هر حرفی که بیشتر از یه بار پشت سر هم اومده رو به یه نمونه کاهش میده، یعنی «خووووب» میشه «خوب». نکته ظریف اینه که باید `r'\1` بنویسیم، نه `r'\1\1`، وگرنه «خوووب» میشه «خوب» که هنوزم اشتباهه.

بعدش توکن‌سازی با `hazm.WordTokenizer()` انجام میشه. بعد از توکن‌سازی کلمات بی‌معنی حذف میشن، ولی اینجا یه نکته مهم هست: Hazm به طور پیش‌فرض کلماتی مثل «خوب»، «عالی» و «بهتر» رو جزو کلمات بی‌معنی حساب کرده! برای تحلیل احساسات این فاجعه‌ست چون دقیقاً همین کلمات بار احساسی دارن. به خاطر همین یه مجموعه کلمات احساسی مهم از لیست بی‌معنی‌های Hazm کنار گذاشتیم تا حذف نشن.

آخرین مرحله هم ریشه‌یابی با `hazm.Lemmatizer()` هستش. خروجی این تابع به شکل «ریشه‌ماضی#ریشه‌مضارع» میاد و ما فقط بخش اول رو نگه می‌داریم. مثلاً «خوردم»، «می‌خورم» و «خورده» همه میشن «خور».

متن اصلی	بعد از پیش‌پردازش
غذاش خیلی خوب بود!!! [ایموجی قلب]	غذا خیلی خوب بود
کیفیت اصلا خووووب نبود، خیلی بد بود	کیفیت اصلا خوب نبود خیلی بد
سرعت ارسال عالییییی بود [ایموجی تأیید]	سرعت ارسال عالی
یه سایت دیگه ببین: https://example.com	یه سایت دیگه ببین
غذا تازه بود و خوشمزه	غذا تازه خوشمزه



پس بعد از پیش‌پردازش، متن‌ها نسبتاً تمیز شدن و آماده تبدیل به بردار عددی هستن.

۳. فاز دوم: Vectorization

خب، حالا سوال اینه که چطور متن رو به اعداد تبدیل کنیم که مدل بتونه باهاش کار کنه؟ سه روش مختلف پیاده‌سازی کردیم که هر کدوم ایده متفاوتی دارن.

۱.۳. Count Vectorizer

ساده‌ترین روش اینه که بشمریم هر کلمه چند بار توی هر نظر اومده. انگار یه جدول بزرگ درست می‌کنیم که سطرهای نظرات و ستون‌های کلمات مختلف هستن و هر خونه تعداد تکرار اون کلمه توی اون نظر رو نگه می‌داره. بهش می‌گن Bag of Words^۵ چون ترتیب کلمات رو نادیده می‌گیره، انگار همه کلمات رو ریختی توی یه کیسه.

تنظیماتی که انتخاب کردیم: حداکثر 50,000 ویژگی، bigram^۶ هم فعال کردیم یعنی علاوه بر کلمات منفرد، جفت کلمات مجاور هم به عنوان ویژگی حساب میشن. مثلاً «نه خوب» یا «اصلاً بد» به عنوان یه ویژگی واحد ثبت میشه که اطلاعات بیشتری نسبت به «نه» و «خوب» جداگانه داره. همچنین ویژگی‌هایی که توی کمتر از 2 سند دیده شدن رو نادیده گرفتیم چون احتمالاً اسم خاص یا غلط نگارشی‌ان.

۲.۳. TF-IDF

TF-IDF یه قدم جلوتر از Count Vectorizer هستش. ایده‌اش اینه که یه کلمه‌ای که توی همه نظرات زیاد میاد احتمالاً اطلاعات خاصی درباره احساس نداره، در حالی که کلمه‌ای که توی یه نظر خاص زیاده ولی توی بقیه کمه، احتمالاً مهم‌تره. مثلاً کلمه «غذا» احتمالاً توی اکثر نظرات هست ولی اطلاعات احساسی خاصی نداره. فرمولش اینه که وزن هر کلمه حاصل ضرب TF (فراوانی کلمه توی سند) در IDF (لگاریتم معکوس تعداد اسناد حاوی اون کلمه) هستش. به علاوه sublinear_tf=True هم گذاشتیم که به جای تعداد خام، از لگاریتمش استفاده کنه تا کلمه‌ای که 50 بار اومده خیلی بیشتر از کلمه‌ای که 5 بار اومده وزن نگیره. خوبیش چیه؟ کلمات رایج و بی‌اطلاعات وزن کمتری می‌گیرن و کلمات تشخیص‌دهنده برجسته‌تر میشن. بدیش چیه؟ وقتی قبلش lemmatization^۷ اجرا کردیم، بخشی از این مزیت از قبل انجام شده.

۳.۳. Word2Vec

Word2Vec یه شبکه عصبی ساده‌ست که یاد می‌گیره هر کلمه رو با یه بردار چگال 100 بُعدی نمایش بده، به طوری که کلمات با معنی مشابه توی فضا به هم نزدیک باشن. انگار داری به هر کلمه یه آدرس 100 بُعدی میدی. مدل رو کاملاً از صفر روی کورپوس خودمون آموزش دادیم، بدون هیچ وزن از پیش‌آموزش‌دیده‌ای. معماری Skip-gram^۸ انتخاب کردیم که برای کورپوس‌های کوچک‌تر معمولاً بهتر جواب میده. برای تبدیل سند به بردار هم میانگین بردار همه کلماتش رو حساب کردیم.

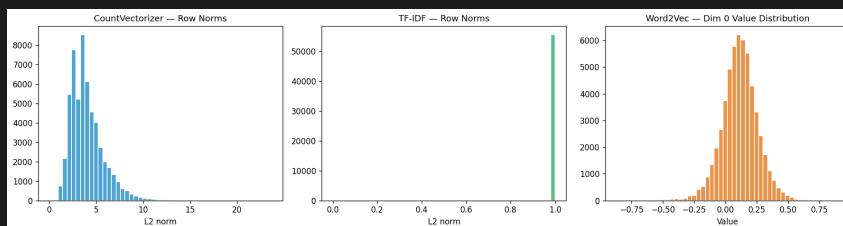
^۵ Bag of Words: مدل نمایش متن که فقط تعداد حضور هر کلمه رو نگه میداره و ترتیب کلمات اصلاً مهم نیست.

^۶ bigram: جفت کلمات مجاور که به عنوان یه ویژگی واحد در نظر گرفته میشن. مثلاً «نه خوب» یه bigramه. در مقابل، کلمات منفرد رو unigram می‌گن.

^۷ lemmatization: فرایند تبدیل هر کلمه به ریشه پایه‌اش. مثلاً «خوردم»، «می‌خورم» و «خورده» همه میشن «خور».

^۸ Skip-gram: یکی از دو معماری اصلی Word2Vec. از یه کلمه یاد می‌گیره کلمات اطرافش رو پیش‌بینی کنه. در مقابل CBOW هستش که از کلمات اطراف، کلمه وسط رو پیش‌بینی میکنه.

بدیش اینه که با میانگین‌گیری ترتیب کلمات و وزنشون از دست میره. مثلاً «خوب نبود» و «نبود خوب» بردار یکسانی میدن. خوبیش اینه که کلمات مشابه معنایی به هم نزدیکن و مدل معناشناسی یاد گرفته.



شکل ۳: توزیع نُرم بردارها در سه vectorizer

پس سه روش داریم با سه دیدگاه متفاوت: CountVectorizer فرکانس خام رو نگه میداره، TF-IDF کلمات مهم‌تر رو برجسته می‌کنه، و Word2Vec معنا رو یاد می‌گیره.

۴. فاز سوم: آموزش و ارزیابی مدل‌های پایه

۱.۴. مدل‌ها و تنظیماتشون

شش تا مدل کلاسیک رو روی هر سه vectorizer آزمایش کردیم. یه نکته فنی مهم قبل از نتایج: MultinomialNB و ComplementNB^۹ با مقادیر منفی کنار نمیان، پس برای ویژگی‌های Word2Vec که ممکنه منفی باشن از GaussianNB استفاده کردیم. همچنین LinearSVC خودش predict_proba^{۱۰} نداره ولی برای Stacking بهش نیاز داریم، پس اون رو داخل CalibratedClassifierCV^{۱۱} گذاشتیم. تنظیمات هر مدل رو توی جدول زیر آوردیم:

مدل	تنظیمات اصلی
ComplementNB / GaussianNB	alpha=0.1
SVM (Linear)	CalibratedClassifierCV داخل C=1.0, max_iter=1000
Decision Tree	max_depth=20, min_samples_leaf=5
Random Forest	n_estimators=200, max_depth=30, n_jobs=-1
KNN	n_neighbors=7, metric=cosine
AdaBoost	n_estimators=100, learning_rate=1.0

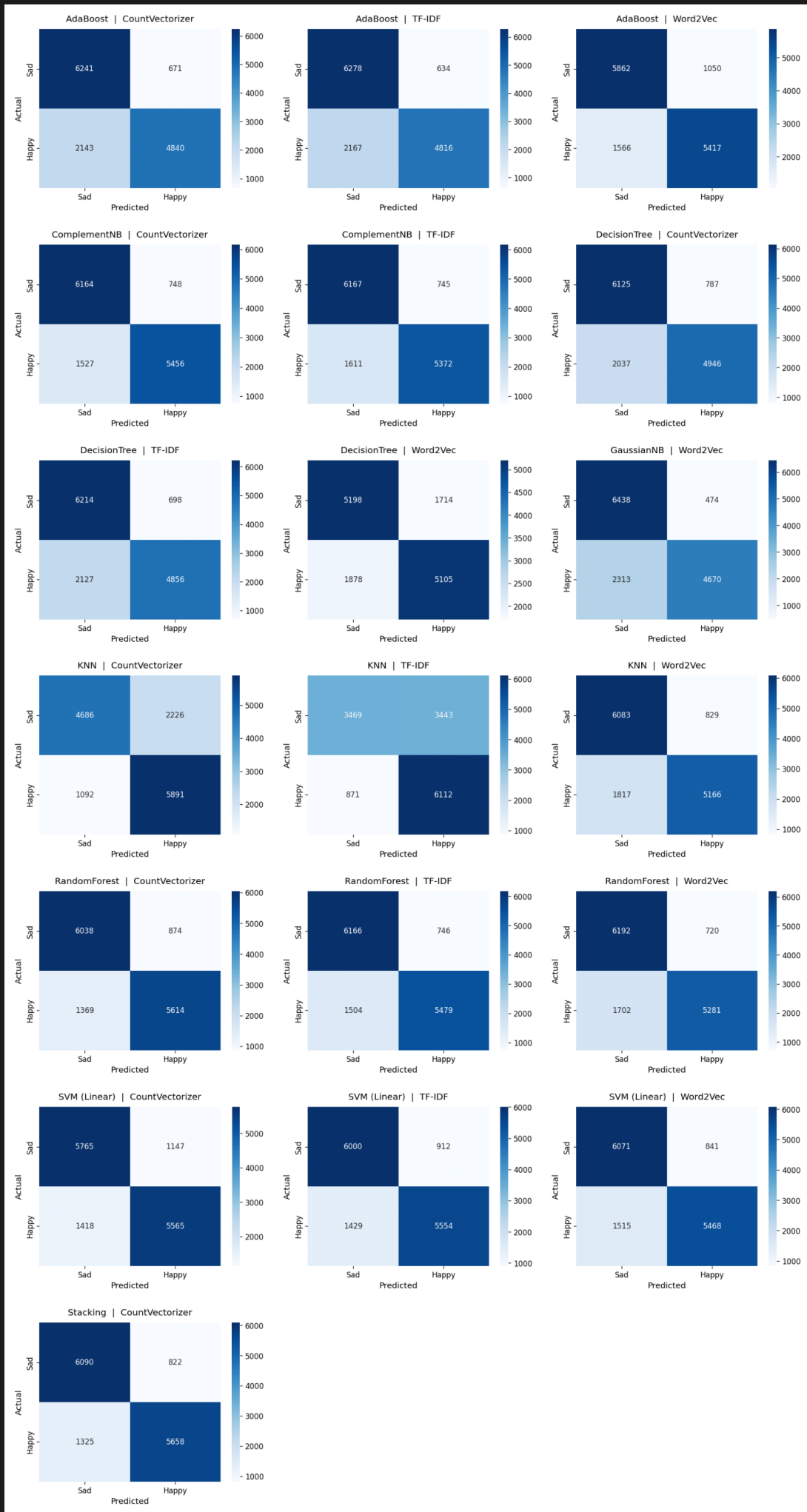
جدول ۲: مدل‌های پایه و تنظیماتشون

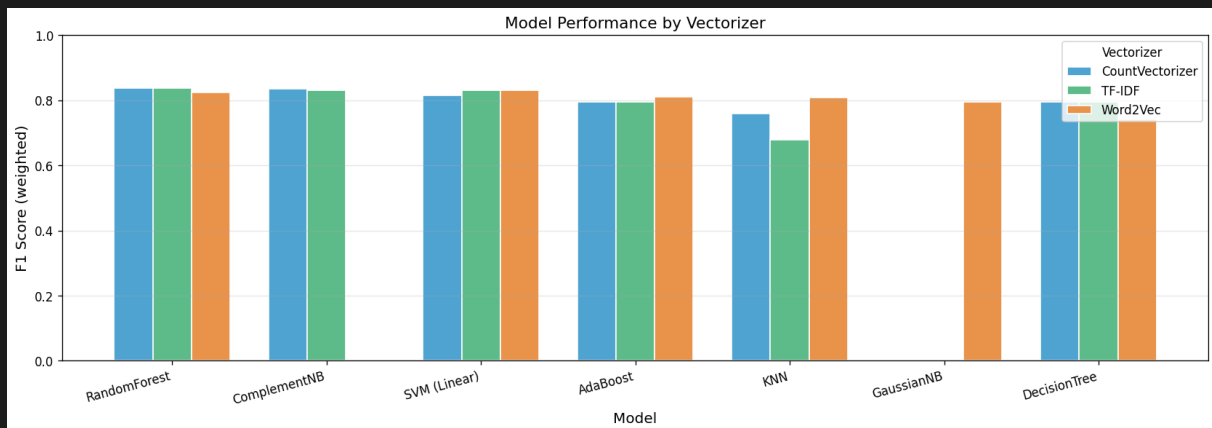
^۹ComplementNB: نوع بهینه‌شده‌ای از Naive Bayes که برای طبقه‌بندی متن طراحی شده. به جای اینکه احتمال یه کلاس رو حساب کنه، احتمال مکملش رو حساب می‌کنه و توی عمل روی متن‌های نامتوازن بهتر جواب میده.
^{۱۰}predict_proba: متدی در scikit-learn که به جای فقط پرچسب کلاس، احتمال تعلق به هر کلاس رو برمیگردونه.
^{۱۱}CalibratedClassifierCV: به این احتمالات نیاز داره تا ورودی متا-مدل رو بسازه.
^{۱۲}CalibratedClassifierCV: یه wrapper در scikit-learn که دور یه مدل میپیچه و بهش قابلیت تولید احتمالات کالیبره‌شده میده، حتی اگه مدل خودش predict_proba نداشته باشه.

۲.۴. نتایج کامل

F1	فراخوانی	صحت	دقت	Vectorizer	مدل
0.8384	0.8386	0.8404	0.8386	CountVectorizer	RandomForest
0.8376	0.8381	0.8422	0.8381	TF-IDF	RandomForest
0.8358	0.8363	0.8406	0.8363	CountVectorizer	ComplementNB
0.8313	0.8315	0.8334	0.8315	TF-IDF	SVM (Linear)
0.8301	0.8304	0.8337	0.8304	Word2Vec	SVM (Linear)
0.8298	0.8304	0.8358	0.8304	TF-IDF	ComplementNB
0.8249	0.8257	0.8325	0.8257	Word2Vec	RandomForest
0.8153	0.8154	0.8159	0.8154	CountVectorizer	SVM (Linear)
0.8115	0.8117	0.8135	0.8117	Word2Vec	AdaBoost
0.8087	0.8096	0.8161	0.8096	Word2Vec	KNN
0.7960	0.7994	0.8222	0.7994	Word2Vec	GaussianNB
0.7961	0.7984	0.8139	0.7984	TF-IDF	AdaBoost
0.7953	0.7975	0.8117	0.7975	CountVectorizer	AdaBoost
0.7952	0.7968	0.8069	0.7968	CountVectorizer	DecisionTree
0.7946	0.7967	0.8100	0.7967	TF-IDF	DecisionTree
0.7595	0.7612	0.7682	0.7612	CountVectorizer	KNN
0.7415	0.7415	0.7417	0.7415	Word2Vec	DecisionTree
0.6782	0.6895	0.7191	0.6895	TF-IDF	KNN

جدول ۳: نتایج کامل 18 آزمایش، مرتب شده بر اساس F1





شکل ۵: مقایسه F1 مدل‌ها روی سه vectorizer

۳.۴. تحلیل نتایج

بهترین مدل Random Forest با CountVectorizer شد با F1 برابر 0.8384. جالبه که ComplementNB هم خیلی خوب جواب داد، F1 برابر 0.8358. درواقع برای داده‌های کیسه‌ای، فرضیه Naive Bayes^{۱۲} آن‌قدرها هم بد نیست.

بدترین نتیجه رو KNN با TF-IDF گرفت، F1 برابر 0.6782. دلیلش اینه که فضای ویژگی TF-IDF خیلی high-dimensional و sparse^{۱۳} هستش و توی این فضا مسئله معروف «نفرین ابعاد» پیش میاد: فاصله‌ها بی‌معنی میشن و KNN دیگه نمیتونه همسایه‌های واقعی رو پیدا کنه.

درخت تصمیم هم نتایج متوسطی داشت چون نمیتونه مرزهای تصمیم‌گیری پیچیده فضاهای پُررُبع رو درست مدل کنه. SVM هم برخلاف انتظار در بعضی جاها ضعیف بود؛ احتمالاً به این دلیل که بعد از lemmatization فضای ویژگی یکنواخت‌تر شده و مزیت SVM در جداسازی خطی کمتر محسوس شده.

۴.۴. نمونه‌هایی از پیش‌بینی‌های درست و اشتباه

متن (پس از پیش‌پردازش)	واقعی	پیش‌بینی
غذا تازه خوشمزه سرعت عالی	مثبت	مثبت ✓
بسته‌بندی افتضاح غذا سرد رسید	منفی	منفی ✓
غذا خوب بود ولی تأخیر داشت	مثبت	منفی ✗
قیمت مناسب کیفیت متوسط	منفی	مثبت ✗

جدول ۴: نمونه‌هایی از پیش‌بینی‌های درست و اشتباه بهترین مدل

موارد اشتباه نشون میده که مدل روی نظرات مبهم ضعیف‌تره، یعنی جاهایی که هم نکته مثبت داره هم منفی. کیسه کلمات نمیتونه «خوب بود ولی تأخیر داشت» رو از «خوب بود» تمیز بده چون ترتیب و ساختار جمله رو نمیبینه.

پس در نهایت از 18 آزمایش، Random Forest با CountVectorizer برنده شد.

^{۱۲} Naive Bayes independence assumption: فرض می‌کنه حضور هر کلمه توی یه سند مستقل از بقیه کلماته. واقعاً درست نیست ولی توی عمل نتایج قابل قبولی میده.

^{۱۳} high-dimensional: بردار خیلی بُعد زیادی داره (اینجا 50,000 بُعد). sparse: از اون 50,000 خونه فقط 20-30 تاشون مقدار غیرصفر دارن و بقیه همه صفرن.

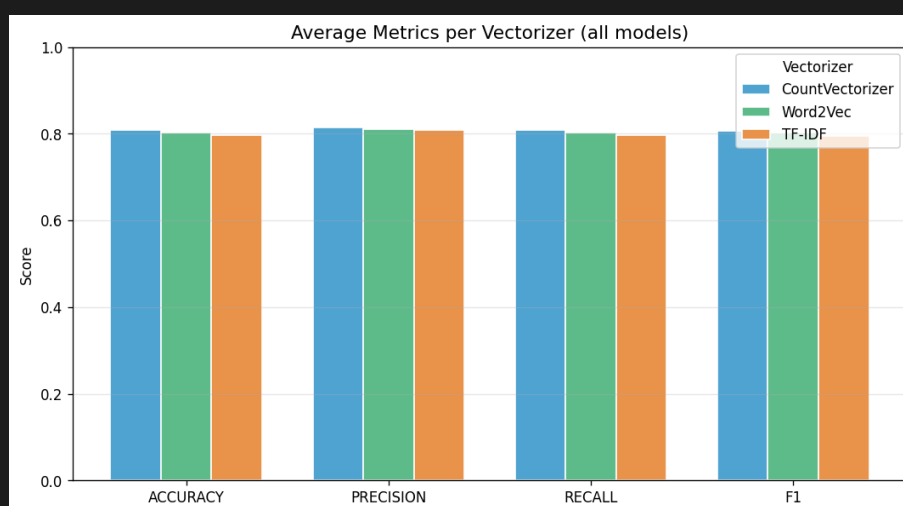
۵. فاز چهارم: انتخاب بهترین Vectorizer

۱.۵. مقایسه میانگین عملکرد

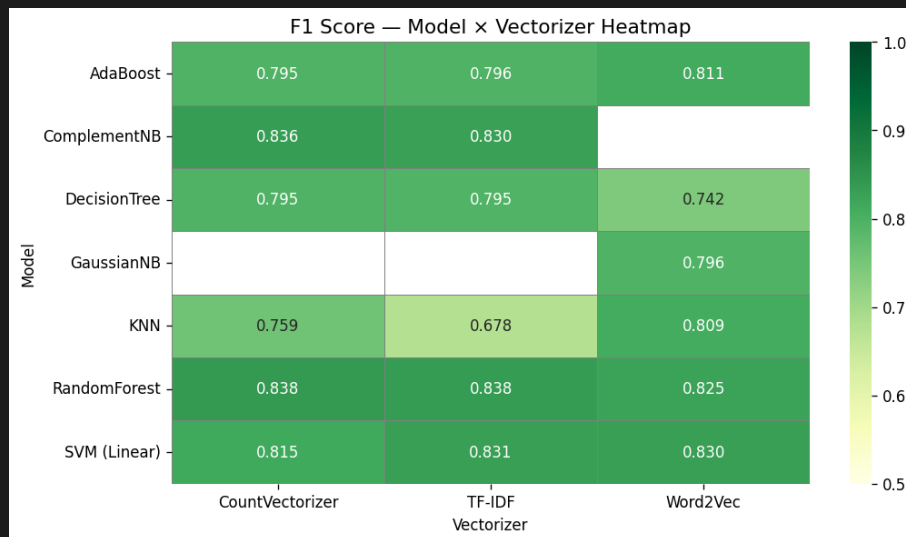
برای انتخاب بهترین vectorizer، میانگین معیارها رو روی همه مدل‌ها حساب کردیم تا به تصویر کلی‌تر داشته باشیم:

Vectorizer	دقت	صحت	فراخوانی	F1
CountVectorizer	0.8076	0.8140	0.8076	0.8066
Word2Vec	0.8030	0.8100	0.8030	0.8021
TF-IDF	0.7974	0.8091	0.7974	0.7946

جدول ۵: میانگین معیارها برای هر vectorizer (روی همه مدل‌ها)



شکل ۶: مقایسه میانگین معیارها در سه vectorizer



شکل ۷: هیتمپ F1 برای ترکیب مدل و vectorizer

۲.۵. چرا CountVectorizer بهتر از TF-IDF شد؟

راستش این نتیجه کمی غیرمنتظره بود چون معمولاً TF-IDF توی طبقه‌بندی متن بهتره. ولی یه توضیح محتمل اینه که lemmatization قبلاً بخشی از کار TF-IDF رو انجام داده. وقتی «خوردم»، «می‌خورم» و «خورده» همه تبدیل میشن به «خور»، فرکانس ریشه‌ها یکنواخت‌تر میشه و دیگه کلمات رایج اون قدر پرتکرار نیستن که وزن‌شون رو کم کنیم. در واقع CountVectorizer با این فرکانس‌های نسبتاً یکسان‌شده، اطلاعات بیشتری برای مدل‌های درختی مثل Random Forest حفظ می‌کنه.

Word2Vec هم با اینکه دوم شد، به خاطر میانگین‌گیری بخشی از اطلاعات ترتیب و وزن کلمات رو از دست میده. 70,000 تا نظر هم برای آموزش یه مدل Word2Vec قوی از صفر زیاد نیست.

پس CountVectorizer رو به عنوان vectorizer فاز بعدی انتخاب کردیم.

۶. فاز پنجم: پیاده‌سازی Stacking

۱.۶. طراحی

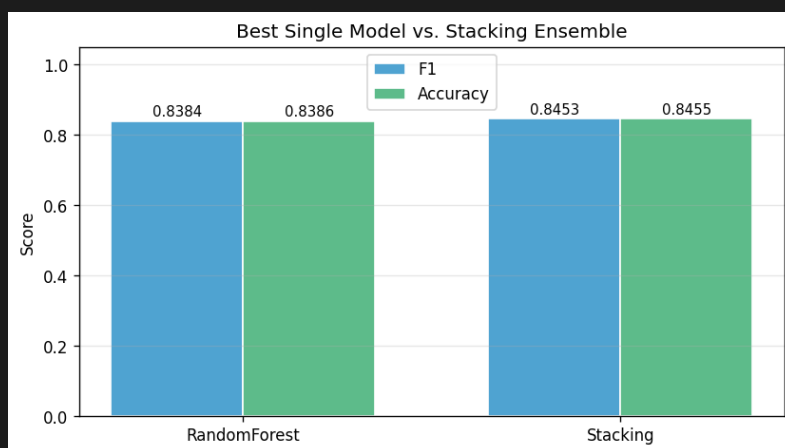
Stacking ایده‌اش اینه که به جای اینکه به یه مدل اعتماد کنیم، چند تا مدل رو آموزش میدیم و بعد یه مدل دیگه (متا-مدل) یاد می‌گیره چطور خروجی‌های اون‌ها رو با هم ترکیب کنه. اگه مدل‌ها روی نوع‌های مختلفی از خطا ضعف داشته باشن، متا-مدل می‌تونه این ضعف‌ها رو جبران کنه.

مدل‌های پایه‌ای که انتخاب کردیم اینان: Decision، Random Forest، SVM (Linear)، ComplementNB و Tree و AdaBoost. متا-مدل هم Logistic Regression با $C=1.0$ هستش. از $cv=5$ استفاده کردیم که متا-مدل روی پیش‌بینی‌های خارج از آموزش ببینه و از نشت داده جلوگیری بشه.

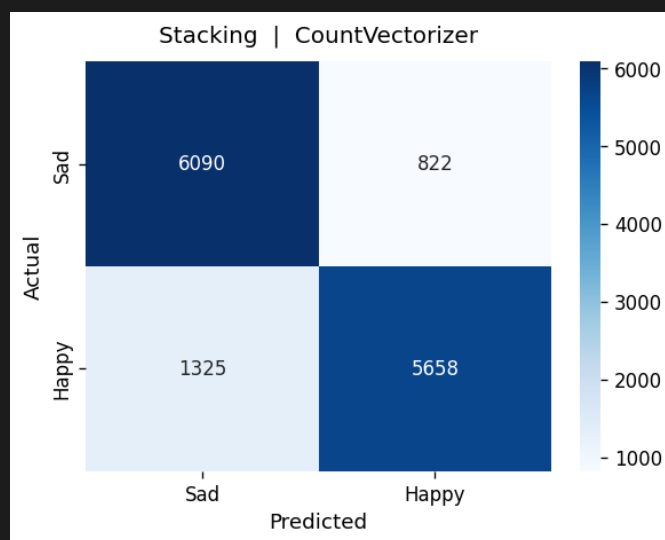
یه نکته فنی که باید بهش دقت کرد اینه که LinearSVC به طور پیش‌فرض predict_proba نداره ولی StackingClassifier بهش نیاز داره. به خاطر همین LinearSVC رو داخل CalibratedClassifierCV گذاشتیم تا احتمالات کالیبره‌شده تولید کنه.

مدل	دقت	صحت	فراخوانی	F1
Stacking Ensemble	0.8455	0.8474	0.8455	0.8453
Random Forest (بهترین تکی)	0.8386	0.8404	0.8386	0.8384

جدول ۶: مقایسه Stacking با بهترین مدل منفرد



شکل ۸: مقایسه عملکرد Stacking در برابر بهترین مدل منفرد



شکل ۹: ماتریس درهم‌ریختگی مدل Stacking نهایی

۳.۶. تحلیل

Stacking با F1 برابر 0.8453 نسبت به بهترین مدل منفرد با F1 برابر 0.8384، به بهبود 0.69٪ داشت. بهبود هست ولی خیلی بزرگ نیست. چرا؟ چند دلیل داره.

اول اینکه Random Forest خودش به روش انسمبل هستش و از قبل خیلی از ضعف‌های مدل‌های ساده رو جبران کرده. دوم اینکه همه مدل‌های پایه روی به فضای ویژگی مشترک آموزش دیدن، یعنی احتمالاً روی

نوع‌های مشابهی از نمونه‌ها اشتباه می‌کنن و تنوع کافی ندارن. Stacking وقتی بیشتر جواب میدن که مدل‌ها روی نقاط قوت مختلف داشته باشن. ولی خب، Stacking بهتر شده. متامدل یاد گرفته در چه مواردی به کدوم مدل پایه بیشتر اعتماد کنه.

۴.۶. نمونه‌هایی از پیش‌بینی‌های درست و اشتباه مدل نهایی

متن (پس از پیش‌پردازش)	واقعی	پیش‌بینی
کیفیت بالا بسته‌بندی عالی سرعت ارسال خوب	مثبت	مثبت ✓
غذا سرد رسید اصلاً تازه نبود ناراضی	منفی	منفی ✓
کمی دیر رسید ولی خوشمزه بود	مثبت	منفی ✗
نه خوب نه بد معمولی	منفی	مثبت ✗

جدول ۷: نمونه‌هایی از پیش‌بینی‌های درست و اشتباه مدل Stacking

پس در نهایت Stacking به قدم جلوتره، ولی نه به جهش بزرگ.

۷. نتیجه‌گیری

خب خلاصه بگیریم چی شد. به پایپ‌لاین کامل برای تحلیل احساسات فارسی درست کردیم روی 70,000 تا نظر Snapfood.

از نظر پیش‌پردازش، مهم‌ترین چیزی که یاد گرفتیم اینه که باید URLها رو قبل از نرمالیزاسیون Hazm حذف کنیم، و اینکه لیست پیش‌فرض کلمات بی‌معنی Hazm شامل کلمات احساسی مهمیه که نباید حذف بشن. از نظر CountVectorizer، vectorization، بهترین نتیجه رو داد که کمی غیرمنتظره بود ولی احتمالاً به خاطر اینه که lemmatization قبلاً بخشی از کار TF-IDF رو انجام داده.

از نظر مدل‌سازی، Random Forest با $F1 = 0.8384$ بهترین مدل منفرد شد. و Stacking با ترکیب پنج مدل این عدد رو به 0.8453 رساند، یعنی بهبود 0.69% داشت.

اگه بخوایم ادامه بدیم، احتمالاً استفاده از BERT^{۱۴} فارسی مثل ParsBERT بهبود خیلی بیشتری میده چون ترتیب کلمات و زمینه جمله رو می‌فهمه، ولی خب اون دیگه کلاسیک نیست.

^{۱۴}BERT (Bidirectional Encoder Representations from Transformers): مدل زبانی بزرگ مبتنی بر Transformer که زمینه دوطرفه جمله رو درک میکنه. ParsBERT نسخه فارسی‌شده‌اش هستش که روی متون فارسی از پیش آموزش دیده.